



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №7
по дисциплине «Разработка баз данных»

Студент группы *ИКБО-66-23 Смирнов А. Ю.*

(подпись)

Принял работу *Копылова Я. А.*

(подпись)

Москва 2025 г.

Оглавление

Цели и задача	3
Выполнение практической работы.....	6
Вывод.....	17

Цели и задача

Цель работы:

Целью данной практической работы является формирование у студентов практических навыков анализа и оптимизации производительности SQL-запросов, а также освоение механизмов управления транзакциями для обеспечения целостности данных (согласно принципам ACID) в СУБД PostgreSQL.

По завершении работы студент должен уметь:

- Сформировать практический навык анализа производительности SQL-запросов с использованием инструмента EXPLAIN ANALYZE.
- Научиться интерпретировать планы выполнения (ПВ), выявляя неэффективные операции, такие как полное сканирование таблицы (Seq Scan).
- Освоить создание различных типов индексов (B-tree, Partial, Function-based), как основного средства для ускорения операций поиска данных.
- Закрепить понимание транзакций как логической единицы работы и освоить использование команд BEGIN, COMMIT и ROLLBACK для обеспечения атомарности операций.
- Научиться моделировать и устранять проблемы параллельного доступа (аномалию «Неповторяемое чтение») с помощью уровней изоляции транзакций (REPEATABLE READ).

Постановка задачи:

Для выполнения практической работы необходимо последовательно выполнить следующие шаги, адаптируя примеры из БД «Аптека» к вашей собственной базе данных:

Подготовка базы данных.

Следуя руководству, наполнить одну из ключевых таблиц вашей БД большим объемом данных (не менее 20 000 строк). Это обязательно для корректной демонстрации работы оптимизатора.

Задание №1: анализ и оптимизация (3 сценария)

Определить три различных «медленных» запроса к вашей БД, которые можно оптимизировать с помощью разных типов индексов (например, стандартный B-Tree, индекс по выражению, частичный/отфильтрованный индекс). Если в вашей базе недостаточно данных, выполните для одной из своих таблиц действия по автоматической генерации содержимого, описанные в разделе «Подготовка базы данных».

Для каждого из 3-х сценариев:

1. Выполнить анализ запроса «КАК ЕСТЬ» (без индекса) с помощью EXPLAIN ANALYZE.
2. Привести план выполнения «ДО», письменно проанализировать его и выявить причину низкой производительности (например, Seq Scan).
3. Создать необходимый INDEX для оптимизации.
4. Повторно выполнить EXPLAIN ANALYZE.
5. Привести план выполнения «ПОСЛЕ», демонстрирующий использование индекса (например, Index Scan).
6. Обязательно сформировать сравнительную таблицу (см. Таблица 1), демонстрирующую разницу в производительности (план, Execution Time) «ДО» и «ПОСЛЕ».

Задание №2: демонстрация атомарной транзакции (COMMIT).

По примеру раздела 2.2 реализуйте в своей базе одну бизнес-операцию (минимум две связанные операции изменения данных) внутри транзакции BEGIN...COMMIT. Задokumentировать все шаги и результаты, сделать выводы.

Задание №3: демонстрация отката транзакции (ROLLBACK).

Адаптировать приведённый в разделе 2.3 SQL-скрипт, моделирующий сбой операции, под свою предметную область, повторив описанные действия. Задokumentировать все шаги и результаты, сделать выводы.

Задание №4: моделирование аномалии «Неповторяемое чтение».

Используя два редактора SQL, смоделировать проблему «неповторяемое чтение» на уровне изоляции по умолчанию (READ COMMITTED) по приведённому в разделе 2.4 образцу. Задokumentировать все шаги и результаты, сделать выводы.

Задание №5: устранение аномалии «Неповторяемое чтение».

Повторить моделирование из Задания №4, но с использованием уровня изоляции REPEATABLE READ. В качестве образца использовать раздел 2.5. Задokumentировать все шаги и результаты, сделать выводы.

Выполнение практической работы

Демонстрация используемых таблиц.

1. Скриншоты всех используемых таблиц индивидуальной схемы данных.

Таблица 1. Таблица client.

	AZ id_client	AZ first_name	AZ last_name	AZ middle_name	AZ driver_license	AZ phone	AZ email	AZ passport_number	registration_date	AZ client_type
1	CL001	Олег	Морозов	Z	77AA123456	+79181112233	oleg@mail.ru	4510123456	2025-09-25	Regular
2	CL002	Лариса	Куришова	[NULL]	77BB654321	+79180001122	larisa@gmail.com	4510987654	2025-09-25	VIP
3	CL003	Сергей	Волков	[NULL]	77CC112233	+79182223344	volkov@yandex.ru	4510567890	2025-09-25	Corporate
4	CL004	Елена	Соколова	[NULL]	77DD445566	+79183334455	sokolova@mail.ru	4510112233	2025-11-06	VIP
5	CL005	Сидоров	Сидоров	[NULL]	7700999888	+79187776655	[NULL]	4510112299	2025-11-06	[NULL]
6	CL006	Иван	Сидоров	[NULL]	7700999288	+79187776455	[NULL]	4510212299	2025-11-06	[NULL]
7	CL007	Алексей	Петров	[NULL]	7700777655	+79189998877	[NULL]	45107778899	2025-11-06	[NULL]

Таблица 2. Таблица service.

	id_service	name	description	price	duration
1	1	Стрижка бороды	Оформление и коррекция бороды	1 000	30
2	2	Мужская стрижка	Классическая мужская стрижка	1 500	45
3	3	Укладка волос	Стрижка и укладка волос для мужчин	1 200	40
4	4	Мужской маникюр	Уход за руками и ногтями для мужчин	800	30
5	5	Уход за лицом	Чистка лица и уход	1 500	60

Таблица 3. Таблица payment_method.

	AZ id_payment_method	AZ method_name	is_active
1	PM001	Кредитная карта	[v]
2	PM002	Дебетовая карта	[v]
3	PM003	Наличные	[v]
4	PM004	Банковский перевод	[v]

Таблица 4. Таблица car_type.

	AZ id_car_type	AZ type_name	seats	luggage_capacity	AZ fuel_type	AZ transmission
1	CT001	Эконом	5		2 Бензин	Механика
2	CT002	Комфорт	5		3 Бензин	Автомат
3	CT003	Бизнес	5		3 Дизель	Автомат
4	CT004	Премиум	5		4 Бензин	Автомат

Таблица 5. Таблица booking_status

	AZ id_booking_status	AZ status_name	AZ description
1	BS001	Ожидание	Бронь ожидает подтверждения
2	BS002	Подтверждена	Бронь подтверждена
3	BS003	Активна	Автомобиль у клиента
4	BS004	Завершена	Автомобиль возвращен
5	BS005	Отменена	Бронь отменена

Таблица 6. Таблица car.

(BETWEEN).

Листинг 1 – Сценарий 1: Шаг «До»

```
-- Листинг 1 – Этап «ДО» индекса
EXPLAIN ANALYZE
SELECT * FROM Test_Cars
WHERE location_id = 1;
```

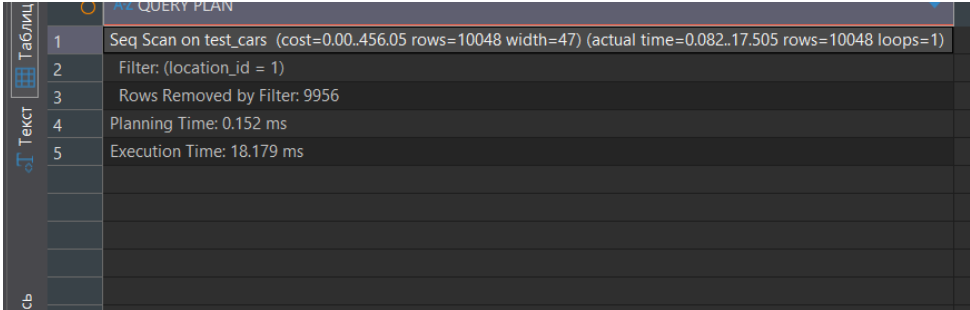


Рисунок 1 – Скриншот результата запроса.

Листинг 2 – Сценарий 1: Создание индекса и шаг «После»

```
-- Листинг 2 – Создание B-Tree индекса
CREATE INDEX idx_test_cars_location ON Test_Cars(location_id);

-- Листинг 3 – Этап «ПОСЛЕ» индекса
EXPLAIN ANALYZE
SELECT * FROM Test_Cars
WHERE location_id = 1;
```

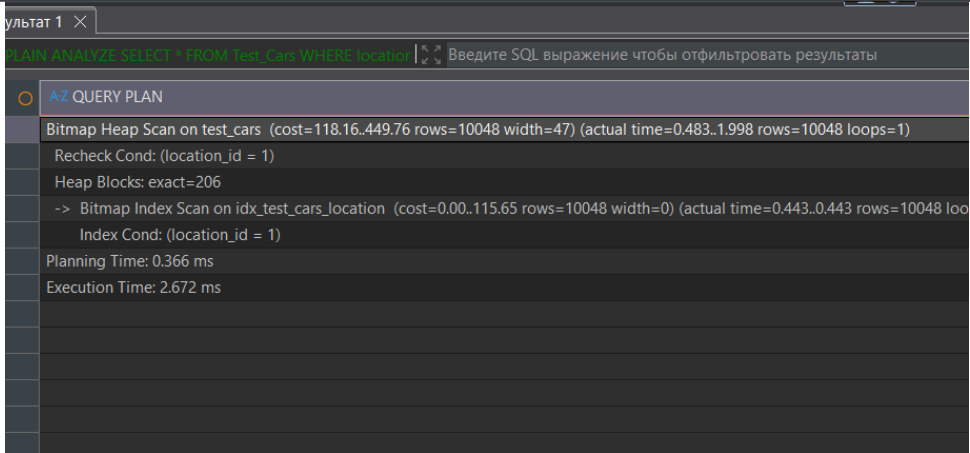


Рисунок 2 – Скриншот выполнения запроса.

Таблица 1 - Сравнение

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Index Scan	Выбор отличается
Execution Time	18.179 мс	2.672 мс	Запрос ускорился примерно в 9 раз

1.2 Сценарий 2. Индекс по выражению (Function-based)

Обычный индекс B-Tree (как idx_medicines_name) не будет использоваться, если вы применяете к столбцу какую-либо функцию в WHERE, например LOWER(). Для решения таких задач, используется индекс по выражению (или «индекс на вычисляемых столбцах»). Он индексирует не сам столбец (name), а результат функции

(в нашем случае – LOWER(name)).

Без этой функции поиск товара «аспирин» вернул бы нам 0 совпадений... Ведь товар в базе сохранён как «Аспирин», а так как коды символов «а» и «А» – разные, то и поиск найдёт только точное совпадение.

Листинг 3 – Сценарий 2: Индекс по выражению (поиск по части email)

```
-- 1. Анализ ДО индекса
EXPLAIN ANALYZE
SELECT * FROM Test_Cars
WHERE LOWER(car_name) like 'автомобиль №100';

-- 2. Создаем индекс по выражению
CREATE INDEX idx_client_email_lower ON client(LOWER(email));

-- 3. Анализ ПОСЛЕ индекса
EXPLAIN ANALYZE
SELECT * FROM Test_Cars
WHERE LOWER(car_name) LIKE 'автомобиль №100';
```

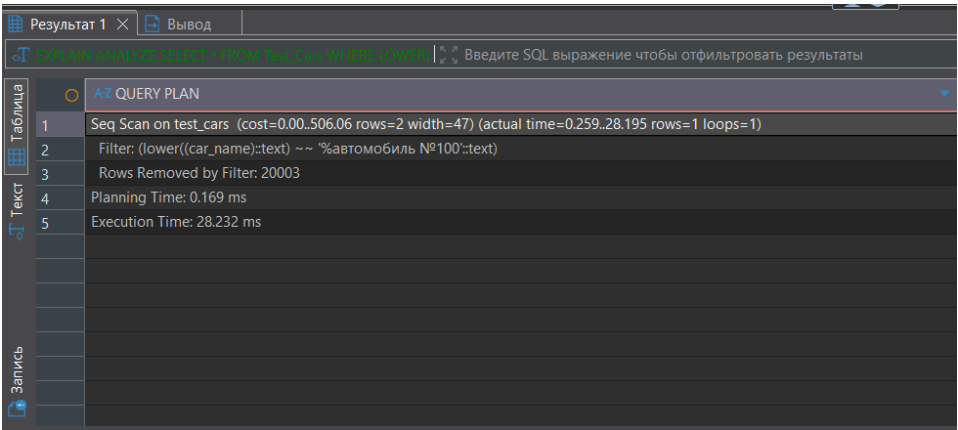


Рисунок 3 – Скриншот запроса «До».

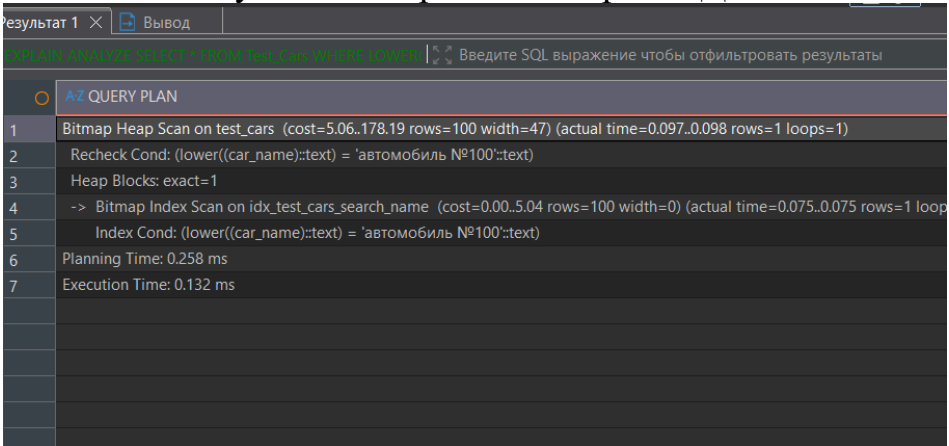


Рисунок 4 – Скриншот запроса «После».

Таблица 2 – Сравнение.

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Bitmap Index Scan	Выбор отличается
Execution Time	28.232 мс	0.132 мс	Запрос ускорился примерно в 213 раз

1.3 Сценарий 3. Частичный (отфильтрованный) индекс

Иногда вы ищете не любые данные, а маленькое, специфическое подмножество (например, `is_archived = true`, или `quantity = 0`). Обычный B-Tree индекс окажется «раздут», храня огромное множество ненужных нам данных. В таких случаях применяется частичный (отфильтрованный) индекс. Он индексирует только те строки, которые соответствуют заданному в WHERE условию, что делает такой индекс очень маленьким и быстрым.

Листинг 4 – Сценарий 3: Частичный индекс для дорогих услуг

```
-- 1. Анализ ДО индекса
EXPLAIN ANALYZE
SELECT * FROM Test_Cars
WHERE mileage = 0

-- 2. Создаем частичный индекс
CREATE INDEX idx_test_cars_zero_mileage ON Test_Cars(id_car)
WHERE mileage = 0;

-- 3. Анализ ПОСЛЕ индекса
EXPLAIN ANALYZE
SELECT * FROM Test_Cars
WHERE mileage = 0;
```

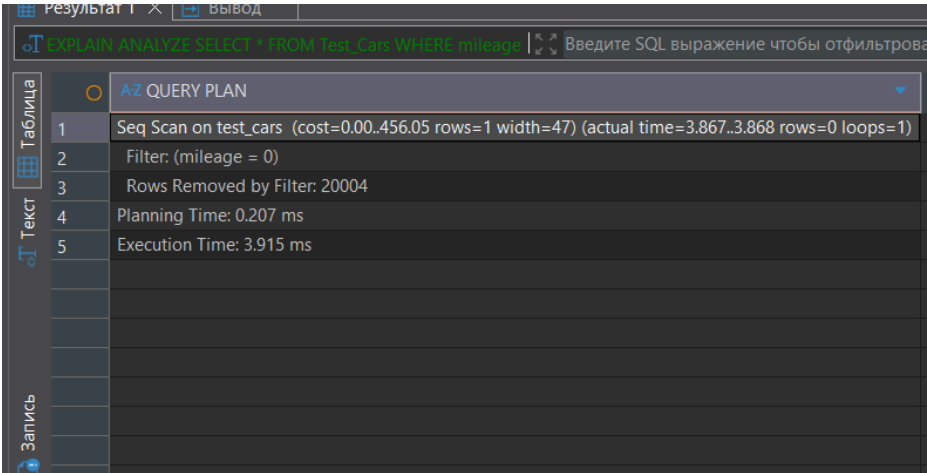


Рисунок 5 – Скриншот запроса «До»

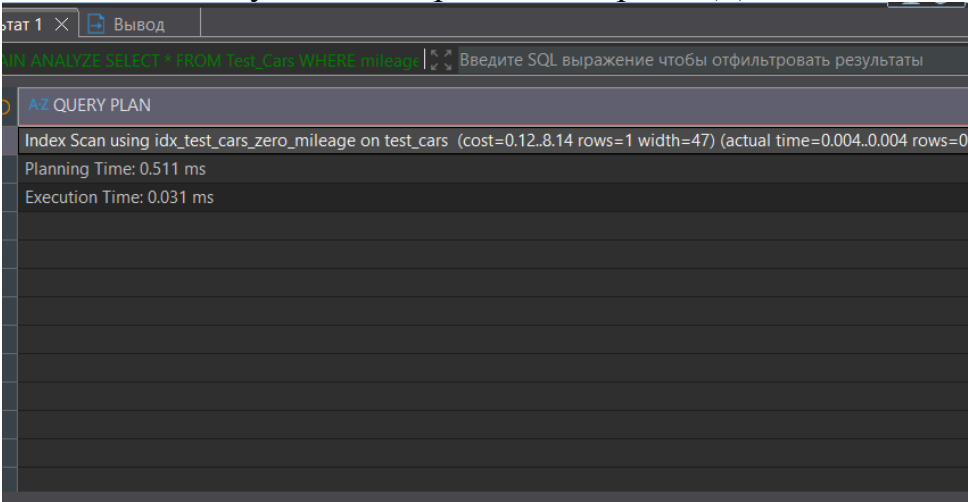


Рисунок 6 – Скриншот запроса «После»

Таблица 3 – Сравнение.

Метрика	До оптимизации	После оптимизации	Вывод
---------	----------------	-------------------	-------

План (Оператор)	Seq Scan	Index Scan	Выбор отличается
Execution Time	3.915 мс	0.031 мс	Запрос ускорился примерно в 126 раза

Задание 2: Атомарная транзакция (COMMIT)

Транзакция — это логическая, неделимая последовательность операций, которая переводит базу данных из одного целостного состояния в другое. Представьте банковский перевод:

- UPDATE users SET balance = balance - 100 WHERE name = 'A';
- UPDATE users SET balance = balance + 100 WHERE name = 'B';

Обе операции должны либо успешно выполниться вместе, либо провалиться вместе. Нельзя допустить, чтобы деньги снялись со счета А, но не дошли до счета Б. Транзакция гарантирует это.

Работа транзакций основана на принципах ACID:

- A (Atomicity / Атомарность) – «Всё или ничего».
- C (Consistency / Согласованность) – БД всегда остается в корректном состоянии.
- I (Isolation / Изолированность) – параллельные транзакции не мешают друг другу.
- D (Durability / Долговечность) – если транзакция завершена, ее изменения постоянны.

```
BEGIN;

-- 1. Создаем заказ (используем car_name вместо id)
INSERT INTO Car_Rental_Orders (
    car_name, client_name, rental_days, daily_rate, total_cost
) VALUES (
    'Автомобиль №100', 'Иван Иванов', 3, 1000.00, 3000.00
)
RETURNING id_order;

-- 2. Создаем платеж
INSERT INTO Car_Payments (id_order, amount, payment_date)
VALUES (currval('car_rental_orders_id_order_seq'), 3000.00, CURRENT_DATE);

-- 3. Обновляем бонусы клиента
UPDATE Client_Bonuses
SET discount_percent = discount_percent + 1
WHERE client_name = 'Иван Иванов';

COMMIT;

SELECT * FROM Car_Rental_Orders ORDER BY id_order DESC LIMIT 1;
SELECT * FROM Car_Payments ORDER BY id_payment DESC LIMIT 1;
SELECT * FROM Client_Bonuses WHERE client_name = 'Иван Иванов';
```

	id_order	car_name	client_name	rental_days	daily_rate	total_cost	order_date
1	1	Автомобиль №100	Иван Иванов	3	1 000	3 000	2025-12-10

Рисунок 7 – Скриншот результата запроса.

Задание 3: Откат транзакции (ROLLBACK)

Происходит следующее:

- Команда UPDATE отрабатывает успешно, но её изменения пока только внутри транзакции (для других пользователей они не видны).
- Команда INSERT вызывает ошибку: нарушено ограничение уникальности/первичного ключа.
- После этой ошибки вся транзакция помечается PostgreSQL как «прерванная» (aborted).
- Команда COMMIT уже не будет выполнена. Соединение остаётся «внутри» ошибочной транзакции.

Важно понимать: PostgreSQL не выполняет за нас автоматический ROLLBACK в явном

```
-- Моделирование сбоя операции
BEGIN;

-- 1. Обновляем цену аренды автомобиля
UPDATE Test Cars
SET daily_rate = daily_rate + 200
WHERE car_name LIKE '%Автомобиль №100%';

-- 2. Пытаемся добавить заказ с уже существующим ID (ошибка)
INSERT INTO Car Rental Orders (
    id_order, car_name, client_name, rental_days, daily_rate, total_cost
) VALUES (
    1, -- Этот ID уже существует!
    'Автомобиль №100', 'Иван Иванов', 3, 1200.00, 3600.00
);

-- Если дошли сюда, фиксируем
COMMIT;
```

) и

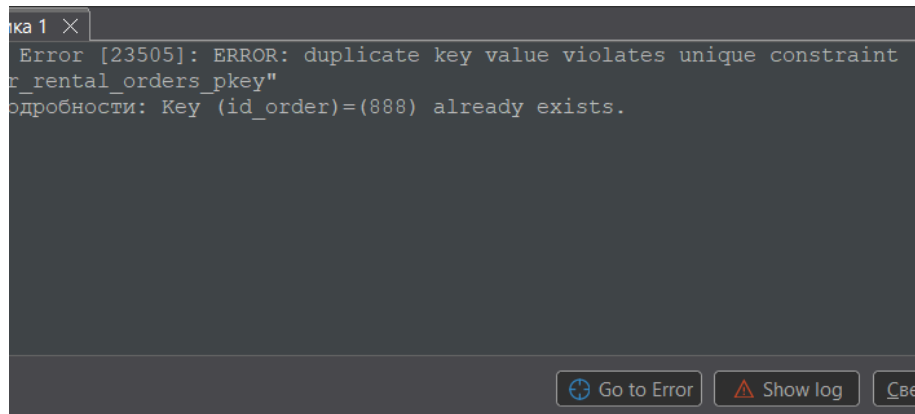


Рисунок 8 – Скриншот результата.

Задание 4: Моделирование аномалии «Неповторяемое чтение»

При параллельной работе нескольких транзакций могут возникать аномалии. «Неповторяемое чтение» (Non-Repeatable Read) — это аномалия, при которой транзакция читает одни и те же данные дважды и получает разные результаты. Это происходит потому, что в промежутке между чтениями другая транзакция успела изменить эти данные и зафиксировать их (COMMIT). Уровень изоляции PostgreSQL по умолчанию, READ COMMITTED («чтение зафиксированного»), допускает эту аномалию.

Листинг 7 – Задание 4: Моделирование аномалии «Неповторяемое чтение» (Скрипт А)

```
BEGIN;  
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
SELECT 'Первое чтение:', price FROM test_service WHERE id = 1;  
SELECT pg_sleep(5);  
SELECT 'Второе чтение:', price FROM test_service WHERE id = 1;  
COMMIT;
```

Листинг 8 – Задание 4: Моделирование аномалии «Неповторяемое чтение» (Скрипт А)

```
-- Меняем цену во время выполнения транзакции в окне 1  
UPDATE test_service SET price = price * 2 WHERE id = 1;  
COMMIT;
```

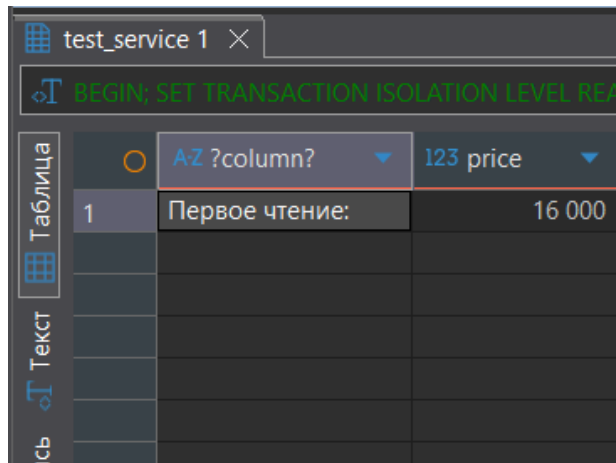


Рисунок 9 – Скриншот результата.

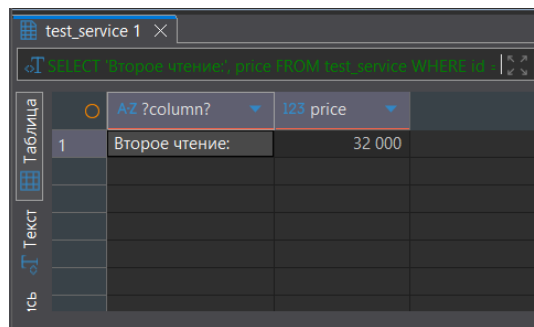


Рисунок 10 – Скриншот результата.

Задание 5: Устранение аномалии с REPEATABLE READ

Для борьбы с этой аномалией используется более строгий уровень изоляции — REPEATABLE READ («повторяемое чтение»).

Как он работает:

На этом уровне транзакция работает с «моментальным снимком» (snapshot) данных, сделанным в момент начала транзакции. Она не видит любые изменения, зафиксированные другими транзакциями (в том числе одиночными запросами) после этого снимка.

```

BEGIN;
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

-- Первое чтение
SELECT price FROM service WHERE id_service = 1;

-- Пауза 10 секунд
SELECT pg_sleep(10);

-- Второе чтение (НЕ увидит изменения из окна 2)
SELECT price FROM service WHERE id_service = 1;

COMMIT;

```

Листинг 10 – Задание 5: Устранение аномалии с REPEATABLE READ (Скрипт Б)

```
UPDATE test_service SET price = price * 2 WHERE id = 1;  
COMMIT;
```

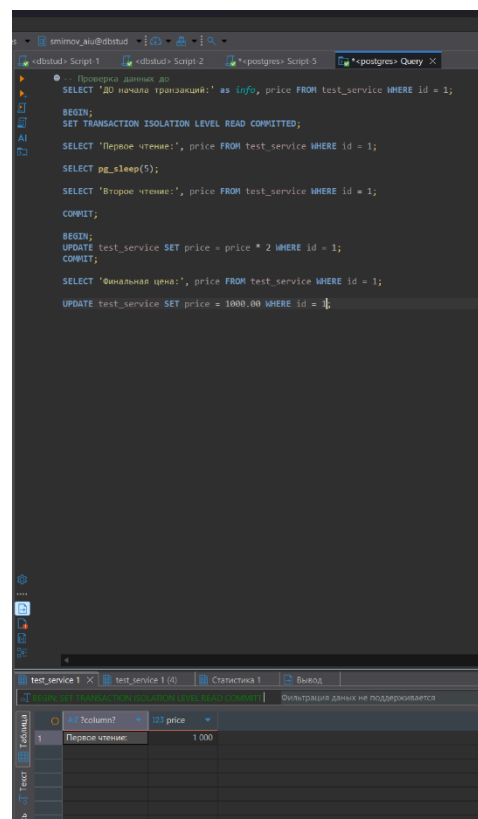


Рисунок 11 – Скриншот результата.

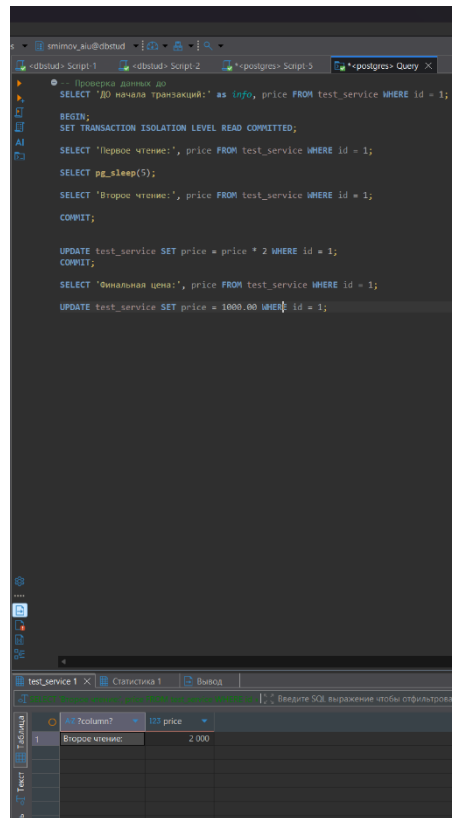


Рисунок 12 – Скриншот результата.

Вывод

В ходе выполнения практической работы по оптимизации запросов и управлению транзакциями в PostgreSQL был получен комплексный опыт работы с реальной базой данных. Были освоены ключевые методы анализа производительности SQL-запросов с использованием EXPLAIN ANALYZE, что позволило наглядно оценить эффективность различных типов индексов. Практика показала, как стратегическое применение индексов — стандартных B-Tree, индексов по выражениям и частичных индексов — кардинально меняет план выполнения запросов, заменяя полное сканирование таблиц на быстрый индексный поиск.